

Unit 2: Overloading

Constructor:

A constructor is a special type of member function of a class which initializes objects of a class. In C++, Constructor is automatically called when object (instance of class) create. It is special member function of the class because it does not have any return type.

- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- It must be placed in public section of class.
- If we do not specify a constructor, C++ compiler generates a default constructor for object (expects no parameters and has an empty body).

Types of Constructors:

1. Default Constructors: Default constructor is the constructor which doesn't take any argument. It has no parameters.

```
// Cpp program to illustrate the
// concept of Constructors
#include <iostream>
using namespace std;
class construct
{
    public:
    int a, b;
    construct()
    {
        a = 10;
        b = 20;
    }
};
int main()
{
    construct c;
    cout << "a: " << c.a << endl
         << "b: " << c.b;
    return 1;
}
```

2. Parameterized Constructors: It is possible to pass arguments to constructors. Typically, these arguments help initialize an object when it is created. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor's body, use the parameters to initialize the object.

```
// CPP program to illustrate
```

```

// parameterized constructors
#include <iostream>
using namespace std;
class Point
{
    private:
    int x, y;
    public:
    Point(int x1, int y1)
    {
        x = x1;
        y = y1;
    }
    int getX()
    {
        return x;
    }
    int getY()
    {
        return y;
    }
};
int main()
{
    Point p1(10, 15);
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    return 0;
}

```

3. Copy Constructor: A copy constructor is a member function which initializes an object using another object of the same class.

A copy constructor is a member function that initializes an object using another object of the same class. A copy constructor has the following general function prototype:

ClassName (const ClassName &old_obj);

```

#include<iostream>
using namespace std;
class Point
{
    private:
    int x, y;
    public:
    Point(int x1, int y1) { x = x1; y = y1; }
    Point(const Point &p1) {x = p1.x; y = p1.y; }
    int getX()           { return x; }
    int getY()           { return y; }
}

```

```

};
int main()
{
    Point p1(10, 15); // Normal constructor is called here
    Point p2 = p1; // Copy constructor is called here
    cout << "p1.x = " << p1.getX() << ", p1.y = " << p1.getY();
    cout << "\np2.x = " << p2.getX() << ", p2.y = " << p2.getY();
    return 0;
}

```

Function Overloading in C++:

Function Overloading in C++ can be defined as the process of having two or more member functions of a class with the same name, but different in parameters. In function overloading, the function can be redefined either by using different types of arguments or a different number of arguments according to the requirement. It is only through these differences compiler can differentiate between the two overloaded functions.

One of the major advantages of Function overloading is that it increases the readability of the program because we don't need to use different names for the same action again and again.

By changing the Number of Arguments

In this way of function overloading, we define two functions with the same names but a different number of parameters of the same type. For example, in the below-mentioned program, we have made two add() functions to return the sum of two and three integers.

```

// first function definition
int add(int a, int b)
{
    cout << a+b;
}

// second overloaded function definition
int add(int a, int b, int c)
{
    cout << a+b+c;
}

```

Here add() function is said to be overloaded, as it has two definitions, one which accepts two arguments and another which accepts three arguments. Which add() function will be called, depends on the number of arguments.

```

int main()
{
    add(10, 20); // add() with 2 parameter will be called
}

```

```

    add(10, 20, 30); //sum() with 3 parameter will be called
}
#include <iostream>
using namespace std;
int add(int a, int b)
{
    cout << a+b << endl;
    return 0;
}
int add(int a, int b, int c)
{
    cout << a+b+c << endl;
    return 0;
}
int main()
{
    add(20, 40);
    add(40, 20, 30);
}

```

In the above example, we overload add() function by changing its number of arguments. First, we define an add() function with two parameters, then we overload it by again defining the add() function but this time with three parameters.

By having different types of Arguments

In this method, we define two or more functions with the same name and the same number of parameters, but the data type used for these parameters are different. For example in this program, we have three add() function, the first one gets two integer arguments, the second one gets two float arguments and the third one gets two double arguments.

```

#include <iostream>
using namespace std;
int add(int x, int y) // first definition
{
    cout<< x+y << endl;

    return 0;
}
float add(float a, float b)
{
    cout << a+b << endl;
    return 0;
}

```

```
double add(double x, double y)
{
    cout << x+y << endl;
    return 0;
}
int main()
{
    add(20, 40);
    add(23.45f, 34.5f);
    add(40.24, 20.433);
}
```

In the above example, we define add() function three times. First using integers as parameters, second using float as parameters and third using double as a parameter. Thus we override the add() function twice.

Function Overloading and Ambiguity

When the compiler is unable to decide which function it should invoke first among the overloaded functions, this situation is known as function overloading ambiguity. The compiler does not run the program if it shows ambiguity error. Causes of Function Overloading ambiguity:

- Type Conversion.
- Function with default arguments.
- Function with a pass by reference

Type Conversion:

```
#include<iostream>
using namespace std;
void function(float);
void function(int);
void function(float x)
{
    std::cout << "Value of x is : " <<x<< std::endl;
}
void function(int y)
{
    std::cout << "Value of y is : " <<y<< std::endl;
}

int main()
{
    function(3.4);
```

```

function(34);
return 0;
}
C:\Users\Lenovo\Desktop\Function_overriding.cpp: In function 'int main()':
C:\Users\Lenovo\Desktop\Function_overriding.cpp:19:17: error: call of overloaded 'function
(double)' is ambiguous
    function(3.4);
              ^

```

The above example throws an error – “call of overloaded ‘function(double)’ is ambiguous”. The function(3.4) will call the first function. The function(34) calls the second function according to our prediction. But this is not what happens because in C++ all the floating-point constants are treated as double not as a float. If we replace the float variable to a double variable, the program will work fine. Therefore we call this a type conversion error from float to double.

Function with Default Arguments:

```

#include<iostream>
using namespace std;
void function(int);
void function(int,int);
void function(int x)
{
    std::cout << "Value of x is : " <<x<< std::endl;
}
void function(int y,int z=12)
{
    std::cout << "Value of y is : " <<y<< std::endl;
    std::cout << "Value of z is : " <<z<< std::endl;
}
int main()
{
    function(12);
    return 0;
}

```

```

C:\Users\Lenovo\Desktop\Function_overriding.cpp: In function 'int main()':
C:\Users\Lenovo\Desktop\Function_overriding.cpp:17:16: error: call of overloaded 'function
(int)' is ambiguous
    function(12);
              ^

```

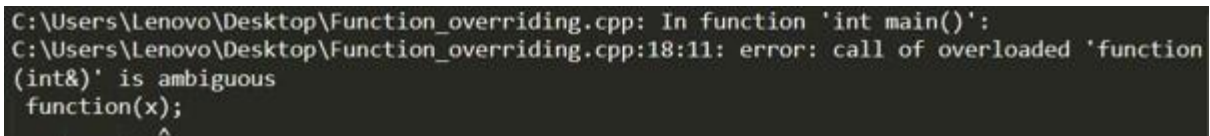
The above example gives an error saying “call of overloaded ‘fun(int)’ is ambiguous”, this is because function(int y, int z=12) can be called in two ways:

1. By calling the function with one argument (and it will automatically take the value of z = 12)
2. By calling the function with two arguments.

When we call the function: `function(12)` we full fill the condition of both `function(int)` and `function(int, int)` thus the compiler gets into an ambiguity shows an error.

Function with pass by reference

```
#include <iostream>
using namespace std;
void function(int);
void function(int &);
void function(int a)
{
    std::cout << "Value of a is : " <<a<< std::endl;
}
void function(int &b)
{
    std::cout << "Value of b is : " <<b<< std::endl;
}
int main()
{
    int x=10;
    function(x);
    return 0;
}
```

A screenshot of a C++ compiler error message. The text is as follows: C:\Users\Lenovo\Desktop\Function_overriding.cpp: In function 'int main()': C:\Users\Lenovo\Desktop\Function_overriding.cpp:18:11: error: call of overloaded 'function(int&)' is ambiguous function(x); ^

The above program gives an error saying “call of overloaded ‘fun(int&)’ is ambiguous”. As we see the first function takes one integer argument and the second function takes a reference parameter as an argument. In this case, the compiler is not able to understand which function is needed by the user as there is no syntactical difference between the `fun(int)` and `fun(int &)` thus it shows an error of ambiguity.

Constructor Overloading:

Constructors can be overloaded in a similar way as function overloading.

Overloaded constructors have the same name (name of the class) but the different number of arguments. Depending upon the number and type of arguments passed, the corresponding constructor is called.

```
// C++ program to demonstrate constructor overloading
#include <iostream>
using namespace std;
class Person {
```

```

private:
    int age;
public:
    // 1. Constructor with no arguments
    Person() {
        age = 20;
    }
    // 2. Constructor with an argument
    Person(int a) {
        age = a;
    }
    int getAge() {
        return age;
    }
};

int main() {
    Person person1, person2(45);
    cout << "Person1 Age = " << person1.getAge() << endl;
    cout << "Person2 Age = " << person2.getAge() << endl;
    return 0;
}

```

This Pointer:

Every object in C++ has access to its own address through an important pointer called **this** pointer. The **this** pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Friend functions do not have a **this** pointer, because friends are not members of a class. Only member functions have a **this** pointer.

```

#include <iostream>
using namespace std;
class Car{
private:
    int Number_of_wheels;
    int Number_of_passengers;
public:
    void getinfo(int x, int y){
        this->Number_of_wheels = x;
        this->Number_of_passengers = y;
    }
    void showinfo(){
        cout<<"Number of Wheels = "<<Number_of_wheels<<"n";
        cout<<"Number of Passengers = "<<Number_of_passengers<<"n";
        cout<<"Address of the current object is = "<<this<<"n"; } };
int main() { Car car; Car *ptr;
ptr=&car; ptr->getinfo(4,5);
ptr->showinfo();
Car car2;

```

```
car2.getinfo(6,8);
car2.showinfo();
return 0;
}
```

The above program gives us a brief understanding of 'this' keyword. 'this' keyword is useful when the variable name is clashing or when multiple objects in a method are involved.

Operators Overloading in C++:

C++ programs can be written without the knowledge of operator overloading. Then too, operator overloading is profoundly used by programmers to make the program intuitive. For example,

We can replace the code like:

```
calculation = add(divide(a, b),multiply(a, b));
```

For equation:

```
calculation = (a/b)+(a*b);
```

Operator overloading provides a simple and easy way for the development of new definitions for most of the operators in C++. With sufficient knowledge, we can almost create a new language of our own by the creative use of the function and operator overloading techniques. We can overload all the operators in C++ except the following:

- Scopeoperator (::)
- Size operator (Sizeof)
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax of Operator Overloading

```
return_type class_name :: operator op(argument_list)
{
// function body
}
```

Where the return type is the type of value returned by the function. class_name is the name of the class.

Implementing Operator Overloading in C++

Operator function must be either non-static (member function) or friend function to get overloaded. Operator overloading function can be applied on a member function if the left operand is an object of that class, but if the Left operand is different, then the Operator

overloading function must be defined as a non-member function. Operator overloading function can be made friend function if it requires access to the private and protected members of the class. For example, the operator op is an operator function where op is the operator being overloaded, and the operator is the keyword. Operator overloading can be achieved by implementing a function that can be either member function, non-member function or friend function.

What is the difference between operator functions and normal functions?

Operator functions are the same as normal functions. The only difference is, the name of an operator function is always operator keyword followed by the symbol of operator and operator functions are called when the corresponding operator is used.

Types of overloading approaches

Operator Overloading can be done by using three approaches, they are

- Overloading unary operator.
- Overloading binary operator.
- Overloading binary operator using a friend function.

Overloading Unary Operator

Let us consider the unary ‘ – ‘ operator. A minus operator when used as a unary it requires only one operand. We know that this operator changes the sign of an operand when applied to a basic data variable. Let us see how to overload this operator so that it can be applied to an object in much the same way as it is applied to an int or float variable. The unary minus, when applied to an object, should decrement each of its data items.

Example:

```
#include <iostream>
using namespace std;
class Height {
public:
    // Member Objects
    int feet, inch;
    // Constructor to initialize the object's value
    Height(int f, int i)
    {
        feet = f;
        inch = i;
    }
    // Overloading(-) operator to perform decrement
    // operation of Height object
```

```

void operator-()
{
    feet--;
    inch--;
    cout << "Feet & Inches after decrement: " << feet << " ' ' " << inch << endl;
}
};

int main()
{
    //Declare and Initialize the constructor of class Height
    Height h1(6, 2);
    //Use (-) unary operator by single operand
    -h1;
    return 0;
}

```

Explanation:

In the above example, we overload ‘ - ’ unary operator to perform decrement in the two variables of Height class. We pass two parameters to the constructor and save their values in feet and inch variable. Then we define the operator overloading function (void operator-()) in which the two variables are decremented by one position. When we write -h1 it calls the operator overloading function and decrements the values passed to the constructor.

Overloading Binary Operator

It is an overloading of an operator operating on two operands. Let’s take the same example of class Height, but this time, add two Height objects h1 and h2.

Example:

```

#include <iostream>
using namespace std;
class Height
{
public:
    int feet, inch;
    Height()
    {
        feet = 0;
        inch = 0;
    }
    Height(int f, int i)
    {
        feet = f;
    }
}

```

```

    inch = i;
}
// Overloading (+) operator to perform addition of
// two distance object using binary operator
Height operator+(Height& d2) // Call by reference
{
    // Create an object to return
    Height h3;
    // Perform addition of feet and inches
    h3.feet = feet + d2.feet;
    h3.inch = inch + d2.inch;
    // Return the resulting object
    return h3;
}
};
int main()
{
    Height h1(3, 7);
    Height h2(6, 1);
    Height h3;
    //Use overloaded operator
    h3 = h1 + h2;
    cout << "Sum of Feet & Inches: " << h3.feet << " " << h3.inch << endl;
    return 0;
}

```

Explanation:

Height operator+(Height &h2), here returns_type of function is class Height thus it returns an object h3 of class Height. In the line h3 = h1 + h2, h1 calls the operator function of its classes objects and takes h2 as a parameter, then we apply h3.feet = feet + d2.feet; and h3.inch = inch + d2.inch; which stores the sum of values of the variables feet and inch in variables associated with the h3 object.

When the statement 'h3 = h1 + h2' invokes the operator overloaded function the object h1 took the responsibility of invoking the function and h2 plays the role of argument that is passed to the function. The above invocation statement is equivalent to h3 = h1.operator+(h2); therefore the data member of h2 are accessed directly and the data member of h2 (that is passed as an argument) are accessed using the dot operator.

Rules for Operator Overloading

- Only the existing operators can be overloaded and new operators cannot be overloaded
- The overloaded operator must contain at least one operand of the user-defined data type.
- We do not use a friend function to overload certain operators. However, the member functions can be used to overload those operators.

- When unary operators are overloaded through a member function they take no explicit arguments, but, if they are overloaded by a friend function they take one argument.
- When binary operators are overloaded through a member function they take one explicit argument, and if they are overloaded through a friend function they take two explicit arguments.

PROGRAMS

1) Function overloading

```
#include <iostream>
using namespace std;
class printData
{
    public:
        void print(int i)
        {    cout << "Printing int: " << i << endl;
        }
        void print(double f)
        {    cout << "Printing float: " << f << endl;
        }
        void print(char* c)
        {    cout << "Printing character: " << c << endl;
        }
};
int main(void) {
    printData pd;
    pd.print(5);
    pd.print(500.263);
    pd.print("Hello C++");
    return 0;
}
```

2) Operator Overloading

```
#include <iostream>

using namespace std;

class Box {

    public:

        double getVolume(void) {

            return length * breadth * height;

        }

}
```

```
void setLength( double len ) {
```

```
    length = len;
```

```
}
```

```
void setBreadth( double bre ) {
```

```
    breadth = bre;
```

```
}
```

```
void setHeight( double hei ) {
```

```
    height = hei;
```

```
}
```

```
Box operator+(const Box& b) {
```

```
    Box box;
```

```
    box.length = this->length + b.length;
```

```
    box.breadth = this->breadth + b.breadth;
```

```
    box.height = this->height + b.height;
```

```
    return box;
```

```
}
```

```
private:
```

```
    double length;    // Length of a box
```

```
    double breadth;   // Breadth of a box
```

```
    double height;    // Height of a box
```

```
};
```

```
int main() {
```

```
    Box Box1;          // Declare Box1 of type Box
```

```
    Box Box2;          // Declare Box2 of type Box
```

```
    Box Box3;          // Declare Box3 of type Box
```

```
    double volume = 0.0; // Store the volume of a box here
```

```
    Box1.setLength(6.0);
```

```
Box1.setBreadth(7.0);  
Box1.setHeight(5.0);  
Box2.setLength(12.0);  
Box2.setBreadth(13.0);  
Box2.setHeight(10.0);  
volume = Box1.getVolume();  
cout << "Volume of Box1 : " << volume <<endl;  
volume = Box2.getVolume();  
cout << "Volume of Box2 : " << volume <<endl;  
Box3 = Box1 + Box2;  
volume = Box3.getVolume();  
cout << "Volume of Box3 : " << volume <<endl;  
return 0;  
}
```